3-Channel LLM Watermarking for Hangul Jamo Structure

github.com/mirulili

1 Background

As the amount of text generated by Large-Scale Language Models (LLMs) rapidly increases, various issues, such as fake news and copyright disputes, are arising. LLM watermarking emerged to address these issues. This technology identifies whether a specific text is a product of AI.

A common method for embedding a watermark in generated text is to adjust the probability that the LLM selects a specific token (Kirchenbauer et al.). The embedded watermark cannot be verified by humans simply by viewing and reading it. Only by detecting subtle patterns using an appropriate detector can one determine whether the product was created by an LLM.

However, existing watermarking approaches have limitations in their general applicability. Specifically, it remains questionable whether the same watermarking method can be applied to texts generated in languages other than English and achieve sufficient performance. This is because mainstream watermarking technologies and their tokenizers are based on English. Representative watermarking methods, such as the Green/Red List Rule (Kirchenbauer et al., 2023) and Google DeepMind's SynthID (Google DeepMind, 2023), were designed based on English datasets.

More fundamentally, the tokenizers used in generative models are also based on English. For example, Byte-Pair Encoding (BPE), the tokenizing algorithm used in the LLaMA model, separates tokens into subword units. However, this approach is not suitable for agglutinative languages such as Korean. Agglutinative languages are languages with complex structures where particles or suffixes are attached to the stem, a prime example of which is Korean. When splitting an agglutinative language into subwords, the meaning contained in the particles or suffixes is inevitably lost. This leads to the possibility of tokens being combined into irregular units when the model generates text.

However, semantic loss isn't the only problem. Current watermarking technologies fail to fully utilize the structural characteristics of Hangul. Hangul is a language with a unique structure where three phonemes—initial consonant, medial vowel, and final consonant—combine to form a single syllable. However, using complete tokens, i.e., token IDs in the form of syllables or words, fails to fully utilize the phoneme-level structure within syllables.

This project focuses on this issue and proposes a new watermarking framework that leverages Hangul's phonological characteristics, namely, the consonant and vowel structures. It begins with the hypothesis that utilizing phoneme units, or "jamo" units, which are smaller than syllables, can increase the amount of information that can be embedded in a watermark.

2 Related Research and Limitations

To specifically define the problems inherent in existing tokenizers, we conducted a simple experiment to check how Korean language tokenized by BPE.

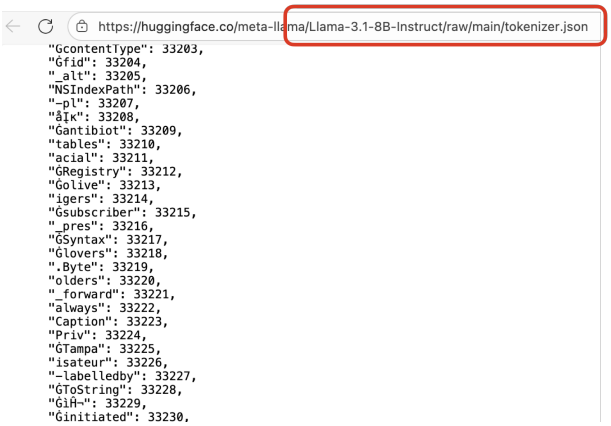

Figure 1: BPE Vocabulary Based on English Words

Based on experimental results, we highlighted the shortcomings of existing watermarking approaches. Kirchenbauer et al., for example, propose a method that segments the vocabulary into Green and Red Lists. They then apply a probabilistic bias to select tokens from the Green List when generating tokens. The token units classified at this time follow the tokenizer's definition. In other words, they depend on the token ID defined before the watermarking step.

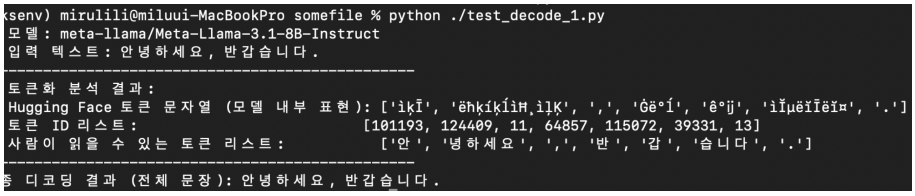

Figure 2: Example of irregular subword segmentation in Korean using LLaMA-3 (BPE) tokenizer. The greeting "안녕하세요" (Hello) is arbitrarily split into "안" and "녕하세요", demonstrating the limitation of BPE for agglutinative languages.

Figure 2 shows the actual tokenization result of the text "안녕하세요, 반갑습니다." (Hello, nice to meet you) using the Meta-Llama-3.1 model. As observed, the word "안녕하세요" is split into arbitrary units like "안" (Token ID: 101193) and "녕하세요" (Token ID: 124409). This segmentation does not align with linguistic morphemes, proving that standard BPE tokenizers fail to capture the structural consistency required for effective watermarking in Korean.

This process ignores contextual and morphological similarities in Korean, which has a different structure from English. For example, if a Korean were asked to segment the phrase "경복궁을" (Gyeongbokgung-eul), it would typically be segmented into units like "경복궁" (Gyeongbokgung) and "으로" (Euro). However, the BPE algorithm splits this phrase into units like "경복" (Gyeongbok) and "궁으로" (Gung-eo). This problem persists even when nouns and particles are well-segmented by chance. Even though "Gyeongbokgung" and "에" are nouns with the same particle, they are recognized as different tokens depending on the particle. This is not due to the meaning of the particle being recognized as a different token; it simply fails to recognize the similarity between "으로" and "에".

Therefore, this project aims to efficiently watermark Korean by decomposing tokens into consonant and vowel units, rather than token IDs. Furthermore, it aims to utilize each of these units—initial, medial, and final consonants—as independent information channels. Ultimately, the goal is to increase watermark information capacity and overcome the limitations of existing methods.

3 Proposed Method (3-Channel Jamo Watermarking)

The core of this project is to manipulate the Logits Processor stage of the tokenizer model. This stage decomposes candidate tokens into consonant units, divides them into three channels, and embeds the watermark.

3.1 Core Principle: Jamo Decomposition and Bit Assignment

Unicode Arithmetic Decomposition is performed to separate syllables into consonant and vowel units. Each Hangul syllable has a corresponding Unicode value. By dividing this by the number of initial, medial, and final consonants, we obtain the indices for each initial, medial, and final consonant. Here, the index refers to the alphabetic order (e.g., ㄱ - 0th, ㄴ - 1st, etc.).

Furthermore, only the last syllable of each token is decomposed. It was determined that separating all syllables of the top-k tokens would incur significant overhead. Next, each separated Jamo index is converted to a bit value through a hash function. The hash function is implemented independently to dynamically adjust its method. In this project, a simple modulo operation is adopted.

This process yields a total of three bit values. For each step of selecting a token, denoted as $step_t$, only one of the three bit values obtained at that step is used. That is, one of three channels—initial, medial, and final—is selected. This value is checked to see if it matches the t-th value in the watermark bitstream. The channel to be used at each stage is sequentially selected in a round-robin fashion.

3.2 Generation Phase

The watermark insertion method adopts an intuitive approach: embedding a string as a bit pattern. For example, let's define the string "LLM" as a watermark and secretly embed it in the text generated by the model. Each character in "LLM" is converted to a byte value, and the pattern, consisting of 8 bits per character, for a total of 24 bits, becomes the desired watermark. Next, this watermark is embedded in the Korean text.

1. **Logits Processor Phase:** When the model predicts the next token, it separates the top k candidate tokens into consonants and vowels. However, only the last syllable of each token is separated. Syllable separation is achieved through Unicode arithmetic decomposition. In Unicode, the Korean character code starts at 0xAC00 ("가"). There are 19 initial consonants, 21 medial vowels, and 28 final consonants. Each syllable, consisting of a combination of these three phonemes, has an index value. If the index of "가" is the Base Index, the index of a syllable is as follows:

$$Index = Base\_Index + (Initial \times 21 \times 28) + (Medial \times 28) + Final \tag{1}$$

   Therefore, by inverting this equation, we can obtain the index of each phoneme.

2. **Target Bit Matching:** In $step_t$, determine whether the t-th watermark bit (target bit) to be inserted matches the hash value of the Jamo index corresponding to the current channel. For example, assume the currently selected channel is the medial vowel. The last syllable of each top-k candidate token is isolated and the medial vowel is selected. The hashed result of the medial vowel index is compared with the target bit. The hashing operation is not significant here, and the equation can be modified by changing the Hash Policy in the future.

3. **Biasing:** If the target bit matches the hash value of the Jamo index, a bias is applied to the token containing that Jamo. Applying a bias increases the Logit value of the token, increasing the probability that it will be selected as a token in the final generated text.

4. **Synchronization:** According to $step_t$, the target bit becomes the t-th bit of the watermark bit string. $Step_t$ is incremented only when watermarking is successful. If no token candidate has a Jamo hash value that matches the target bit, $step_t$ is not incremented and the corresponding step is skipped.

3.3 Detection Phase

Watermark detection assumes that the detector knows the watermark in advance. The detection process is as follows:

1. **Re-tokenization:** The text to be detected is input. The text is re-tokenized using an internal function in the transformers library.

2. **Sequential Verification:** Separate the Jamo units of the last syllable of the token in the exact same manner as generation. Calculate the hash value of the corresponding Jamo index. Each token that constitutes the text is examined and the extracted payload is completed. Note that if the text was generated by Model A, it must be re-tokenized using the same tokenizing algorithm used by Model A. This allows for finding a token array with units exactly equivalent to the specific tokenizer's vocabulary.

3. **Message Reconstruction:** The extracted payload is divided into 8-bit chunks, and each 8-bit chunk is calculated as a 1-byte value. The message is reconstructed by finding the ASCII character that matches each byte value.

4 Experiments and Results

A series of experiments were conducted to verify the effectiveness of this design method. The experiments used the `kogpt2-base-v2` model, a lightweight model optimized for Korean.

```
[Generated Watermarked Text]
산에서 우는 작은 새야, 쭝꾸는 진귀한 새끼, 그리고 쭝꾸는 사오라기 굠을 닮아서 그렇게 되었다.
그래서 진달래꽃밭이나 노적산 등등 온통 진드 권장 제목이다.
지금까지도 그 알 만한 사람들은 진달래를 목격하지 못했다는 말들이 가슴에 남아 있다.
그러나 이 백일몽만한 진달래. 그것은 진심으로 자기 생각하고 자꾸 번득이는 꿀밤이었던 듯싶다.
진짜 진달래밭과 흰 진달래밭으로 가려면 꼭 진달래꽃숲과 노적산과 같은 진달래밭으로 가려면 꼭 노적산, 흰 진달래숲과 노
적산 등등 진달래꽃을 보기 위해 가는 곳마다 산과 꽃 사이를 산책해야 한다는 것이다.
물론 이 진달래밭과 노적산도 결국 진달래숲과 진달래밭이라는 자연의 맛을 생생하게 체험할 수 있는 하나의 생태숲으로 꾸
며져 있어야 한다.
진달래꽃은 덤불을 수 놓을 줄 아는 매의 속성인 것이다.
꽃을 솎아 낼 수도 있는 것은 들을 수 있는 겨를이 없는 뱃신이다.
꽃밭에 비 오는 날
------------------------------

--- 2. Watermark Detection Phase ---
Extracted Payload (bits): '0101001001100101011000010110010000100000010011010110010100100000010010010110011000100000
1011001011011110111010100100000010000110110000101101110'
Accuracy: 100.0%
Z-Score:  14.70
Recovered Message: 'Read Me If You Can'

--- 3. Verification ---
[Verification Success] Watermark Confirmed.
[Recover Success] Original message was successfully verified.
```

Figure 3: A Test Result of Embedding Watermark

```
--- 1. Watermark Generation Phase ---
Original Message: 'Read Me If You Can'
Step 0 Success: 쭝
Step 1 Success: 꾸
Step 2 Success: 는
Skip (Mismatch)
Step 3 Success: 귀한
Step 4 Success: 새
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
Skip (Mismatch)
```

Figure 4: Finding Target-Matching Tokens

4.1 Watermark Insertion and Reconstruction Performance (Capacity, Accuracy)

For convenience, the watermark to be inserted was composed of alphabets that can be easily converted to ASCII. The prompt was defined and then the model was asked to generate text. The watermark restoration rate depended on the bias value and the number of generated tokens.

A higher watermarking bias value ensured that the watermark was fully embedded within the specified number of tokens generated. Consequently, restoration achieved 100% accuracy. Conversely, a lower bias value resulted in more tokens being selected that did not match the target bits, preventing the watermark from being fully embedded within the specified number of tokens.

Through a series of tests, the bias value was set to 5.0 and the number of generated tokens to 200. This process proved that the project's character decomposition logic functioned accurately.

4.2 Robustness

Next, to verify robustness, we assumed an attack scenario against the watermark. The attack involved randomly deleting words from the generated sentence, and the restoration rate was determined. Only one word was selected and deleted from the text generated using watermarking, as even a single word occupies a significant portion of the bitstream when converted to bits.

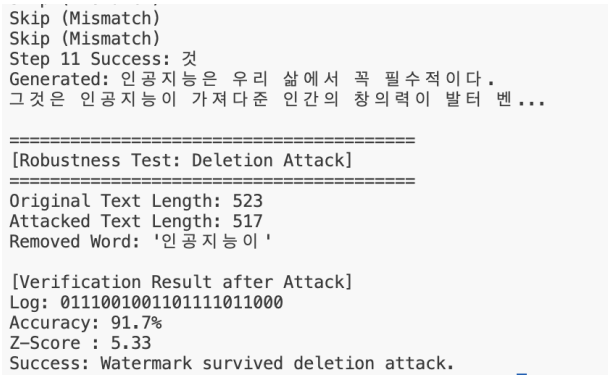

Figure 5: A Test Result of Deletion Attack

Experimental results showed that even after deleting words, some of the watermarked message could be successfully recovered. This is because the consonant bit values were distributed and stored within the remaining tokens. In this experiment, a watermark pattern recovery of more than 50% was considered successful. This indicates that the fine consonant-level patterns were firmly embedded throughout the text.

4.3 Perplexity

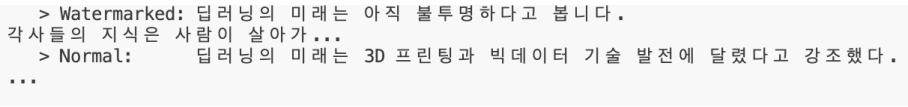

Figure 6: Comparing Watermarked Text and Unwatermarked Text

To evaluate the quality and naturalness of the generated text, we measured the Perplexity (PPL) using the test set. The results are summarized in Table 1.

As shown in the table, the Perplexity increased from 17.61 to 40.61 after applying the watermark. This degradation in text quality is attributed to the high fixed bias value (5.0) used to

Condition	Perplexity (PPL)
Baseline (No Watermark)	17.61
Watermarked (Bias = 5.0)	40.61

Table 1: Comparison of Perplexity between Baseline and Watermarked Text

ensure a 100% detection rate. The strong bias forces the model to select tokens that satisfy the Jamo hash constraints, even if they are less probable in the given context.

This trade-off between detection accuracy and text quality is a known challenge. To mitigate this issue, we proposed the *Context-Based Adaptive Bias Adjustment* method in Appendix C, which dynamically adjusts the bias based on the entropy of the token distribution.

4.4 MarkLLM Evaluation and Visualization

To further validate the detection performance and visualize the watermark distribution, we utilized the MarkLLM framework. Figure 9 illustrates the comparison between watermarked and unwatermarked text distributions.

In the visualization, tokens are color-coded based on their classification by the detector. 'Green Tokens' represent tokens that align with the watermark pattern (biased), while 'Red Tokens' indicate non-matching tokens. As observed in Figure 7, the watermarked text exhibits a distinct pattern of token distribution compared to the unwatermarked text in Figure 8.

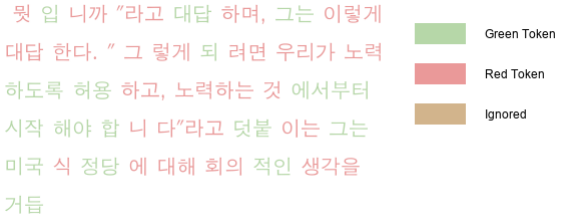

Figure 7: Watermarked Text

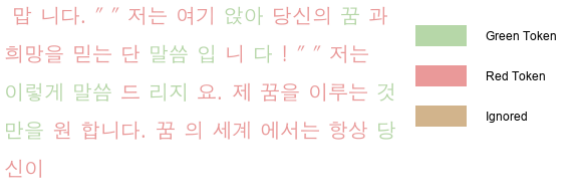

Figure 8: Unwatermarked Text

Figure 9: Visualization of Token Distribution using MarkLLM. The difference in color distribution demonstrates the detector's ability to distinguish watermarked content.

Quantitatively, we evaluated the robustness of the watermark using the MarkLLM framework. The results, as detailed in Table 2, show a True Positive Rate (TPR) of 0.695 and an F1 Score of 0.82. These metrics indicate that the 3-Channel Jamo Watermarking method maintains a reliable detection capability even when evaluated under the rigorous standards of the MarkLLM framework. The high F1 score suggests a balanced performance between precision and recall, validating the effectiveness of embedding information at the phoneme level.

Metric	Value
True Positive Rate (TPR)	0.695
F1 Score	0.820

Table 2: Robustness Evaluation Results using MarkLLM Framework

5 Conclusion

This project implemented a Korean-specific, three-channel consonant-based LLM watermarking system targeting the Korean language. This is significant in that it presents a unique watermarking method that utilizes the unique structure of consonants and vowels as the watermarking unit. This approach ensures that the linguistic characteristics of the Korean language are reflected in the system design.

While AI technology is primarily focused on English-speaking countries, this project demonstrates the performance and quality of watermarking by leveraging the characteristics of non-English languages, presenting a new alternative. We anticipate that this project will serve as a reliable solution to ensure transparency in Korean generative AI in the future.

References

[1] Bryce et al. 딥 러닝을 이용한 자연어 처리 입문. https://wikidocs.net/book/2155

[2] Dathathri, S. et al. (2024). Scalable watermarking for identifying large language model outputs.

[3] DeepMind. (2023). SynthID: Watermarking for AI-Generated Content. Google DeepMind Technical Report.

[4] Gibney, E. (2024). Google unveils invisible 'watermark' for AI-generated text. *Nature*. https://www.nature.com/articles/d41586-024-03462-7

[5] Google. Gemini API. https://ai.google.dev/gemini-api/docs/tokens

[6] Google. SentencePiece. GitHub. https://github.com/google/sentencepiece

[7] Google DeepMind. synthid-text. GitHub. https://github.com/google-deepmind/synthid-text

[8] Google DeepMind. SynthID website. https://deepmind.google/science/synthid/

[9] Hugging Face. LLaMA. Transformers documentation. https://huggingface.co/docs/transformers/v4.42.4/model_doc/llama

[10] Hugging Face. LLM Course: Chapter 6. https://huggingface.co/learn/llm-course/chapter6/

[11] Hugging Face. Introducing SynthID Text. Hugging Face Blog. https://huggingface.co/blog/synthid-text

[12] Kirchenbauer, J. et al. (2023). A Watermark for Large Language Models. arXiv:2301.10226.

[13] Kudo, T., & Richardson, J. (2018). SentencePiece: A Simple and Language-Independent Subword Tokenizer. EMNLP.

[14] OpenAI. tiktoken. GitHub. https://github.com/openai/tiktoken

[15] Park, K. et al. (2020). An Empirical Study of Tokenization Strategies for Various Korean NLP Tasks.

[16] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.

[17] Unicode Consortium. (2023). The Unicode Standard: Hangul Syllable Composition.

A Future Improvements

A.1 Fallback Mechanism for Resolving Watermark Conflicts

In our proposed design, if no tokens with Jamo hash values matching the target bits were found, the $step_t$ was skipped without increasing. However, in actual generation, the Top-k candidates can become fixed on a specific Jamo pattern in certain contexts. This can lead to a deadlock situation where the watermark is not inserted at all.

To prevent this, we are planning to integrate the following fallback policy:

- If the same bit matching failure occurs N consecutive times (e.g., 10 times), the bit is forcibly skipped and the next bit is moved to the next bit.
- The location of the skipped bit is recorded in the metadata and is omitted in the detection phase as well.
- Independent failure counters are provided for each channel (initial/medial/final consonant) to detect recurring failure patterns in specific channels.

This mechanism prevents watermark insertion stalls in real-world environments, minimizes synchronization issues during restoration, and improves debugging convenience.

A.2 Dynamic Channel Weight Adjustment Algorithm

While the Round Robin method is structurally simple, it presents the following challenges due to the characteristics of the Korean language: the high number of syllables without final consonants reduces the usability of the final consonant channel; the pattern of initial and medial consonants becomes excessively monotonous in certain contexts; and the asymmetric distribution of watermark insertion success rates across channels.

Therefore, we aim to introduce a probabilistic channel selection method based on the success rate of each channel. For each channel, the (number of successes) / (number of attempts) is calculated in real time. The weights of channels with low success rates are increased so that these channels are selected more frequently. This prevents the overall payload from being biased toward specific channels and maintains an even distribution during detection. This method is expected to mitigate the syllable structure bias that occurs in Korean sentences and increase the overall watermark insertion rate.

A.3 Context-Based Adaptive Bias Adjustment

In the proposed model, the bias value was fixed at 5.0, but this leads to the following issues depending on the context: In situations with very low entropy (e.g., "안녕하세" → "요"), quality deteriorates significantly, and in situations with high entropy, the embedding rate actually decreases.

To address this, we plan to introduce a dynamic bias adjustment algorithm based on Shannon Entropy:

- Calculate the entropy H of the top-k distribution.
- High H → Increase the bias (increase the embedding rate).
- Low H → Reduce the bias (maintain text quality).

In other words, by automatically adjusting the trade-off between watermark embedding and naturalness according to the situation, we can achieve better results in perplexity experiments.

A.4 Hash Policy Extension and Verification Framework

The current system uses a simple "modulo"-based hash, but the distribution of the hash function directly affects the following: bit distribution uniformity, collision probability, and bias at specific character indices. Therefore, this study aims to design a multi-hash policy verification system. Because the hash policy is separated based on interfaces, the cost of adding a new hash function is very low. This framework provides important quantitative results that can be included in the experimental section (Chapter 4).

A.5 Selective Recovery Strategy — An Alternative to ECC

When using ECC, our model have repeatedly reported the following issues: high insertion failure rates and poorly distributed bit structures throughout the text, which prevent ECC from recovering properly.

Thus, instead of maintaining the complexity of ECC, we would like to adopt the following selective recovery strategy: Repetition Coding.

- Example: Repeating insertion of only the first 8 bits of a 24-bit watermark three times.

This increases the overall payload, but ensures a high recovery rate for important information (e.g., source identifier).

The detection phase uses a Hamming Distance-based nearest neighbor search. The detector assumes the watermark is known: the extracted bitstream E, and the possible original watermark candidates W_i. For all i, if $d(E, W_i) \leq \tau$, W_i is determined as the final watermark. This approach provides practical robustness even without ECC.